AI Assisted Coding Lab 21.3

Veekshitha Adharasani

2303A52175

Batch – 41

Question 1: AI-Assisted Pair Programming

Prompt:

Give code for Expense Tracker.

Code:

```python
import json
import os
class ExpenseTracker:
    def __init__(self, filename='expenses.json'):
        self.filename = filename
        self.expenses = []
        self.load_from_file()
    def add_expense(self, amount, category, description):
        try:
            amount = float(amount)
            if amount <= 0:
                raise ValueError("Amount must be positive")
            self.expenses.append({
                'amount': amount,
                'category': category,
                'description': description
            })
            self.save_to_file()
            print("Expense added successfully!")
        except ValueError as e:
            print(f"Error: {e}")
    def view_expenses(self):
        if not self.expenses:
            print("No expenses recorded yet.")
            return
        print("Expenses:")
        for i, exp in enumerate(self.expenses, 1):
            print(f"{i}. ${exp['amount']:.2f} - {exp['category']} - {exp['description']}")
    def total_expenses(self):
        total = sum(exp['amount'] for exp in self.expenses)
        print(f"Total expenses: ${total:.2f}")
        return total
    def save_to_file(self):
        with open(self.filename, 'w') as f:
            json.dump(self.expenses, f, indent=4)
    def load_from_file(self):
        if os.path.exists(self.filename):
```

```python
class ExpenseTracker:
    def save_to_file(self):
            json.dump(self.expenses, f, indent=4)
    def load_from_file(self):
        if os.path.exists(self.filename):
            try:
                with open(self.filename, 'r') as f:
                    self.expenses = json.load(f)
            except json.JSONDecodeError:
                print("Error loading expenses file. Starting fresh.")
def main():
    tracker = ExpenseTracker()
    while True:
        print("\nExpense Tracker Menu:")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Total Expenses")
        print("4. Exit")
        choice = input("Choose an option: ").strip()

        if choice == '1':
            amount = input("Enter amount: ").strip()
            category = input("Enter category: ").strip()
            description = input("Enter description: ").strip()
            tracker.add_expense(amount, category, description)
        elif choice == '2':
            tracker.view_expenses()
        elif choice == '3':
            tracker.total_expenses()
        elif choice == '4':
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe
acker.py

Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Exit
Choose an option: 1
Enter amount: 5000
Enter category: grocerry
Enter description: Household things
Expense added successfully!

Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Exit
Choose an option:
```

Prompt:

Write a Python program for a simple Expense Tracker Application that allows a user to record daily expenses along with categories (like food, travel, etc.). First, generate a basic version of the program with simple input and storage. Then review the code and identify its limitations, such as lack of input validation or structure. After that, improve the program by adding features like monthly expense summary, proper input validation, and organizing the code into functions for better readability. Finally, provide both the initial and improved versions and explain how the changes make the program more efficient and user-friendly.

Code:

```python
import json
import os
from datetime import datetime
from collections import defaultdict

# Improved Expense Tracker
# With input validation, functions, monthly summary, and file storage

def load_expenses(filename='expenses.json'):
    if os.path.exists(filename):
        try:
            with open(filename, 'r') as f:
                return json.load(f)
        except json.JSONDecodeError:
            print("Error loading expenses file. Starting fresh.")
    return []

def save_expenses(expenses, filename='expenses.json'):
    with open(filename, 'w') as f:
        json.dump(expenses, f, indent=4)

def add_expense(expenses):
    while True:
        amount_str = input("Enter amount: ").strip()
        try:
            amount = float(amount_str)
            if amount <= 0:
                print("Amount must be a positive number.")
                continue
            break
        except ValueError:
            print("Invalid amount. Please enter a number.")

    category = input("Enter category (e.g., food, travel): ").strip()
    if not category:
        category = "misc"
```

```python
def add_expense(expenses):

    category = input("Enter category (e.g., food, travel): ").strip()
    if not category:
        category = "misc"

    description = input("Enter description: ").strip()

    date_str = input("Enter date (YYYY-MM-DD) or press Enter for today: ").strip()
    if not date_str:
        date = datetime.now().strftime('%Y-%m-%d')
    else:
        try:
            datetime.strptime(date_str, '%Y-%m-%d')
            date = date_str
        except ValueError:
            print("Invalid date format. Using today's date.")
            date = datetime.now().strftime('%Y-%m-%d')

    expenses.append({
        'amount': amount,
        'category': category,
        'description': description,
        'date': date
    })
    print("Expense added successfully!")

def view_expenses(expenses):
    if not expenses:
        print("No expenses recorded.")
        return
    print("Expenses:")
    for i, exp in enumerate(expenses, 1):
        print(f"{i}. {exp['date']} - ${exp['amount']:.2f} - {exp['category']} - {exp['description']}")

def total_expenses(expenses):
    total = sum(exp['amount'] for exp in expenses)
```

```python
def monthly_summary(expenses):
    for exp in expenses:
        month = exp['date'][:7]  # YYYY-MM
        monthly[month] += exp['amount']
    print("Monthly Summary:")
    for month, total in sorted(monthly.items()):
        print(f"{month}: ${total:.2f}")

def main():
    expenses = load_expenses()
    while True:
        print("\nExpense Tracker Menu:")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Total Expenses")
        print("4. Monthly Summary")
        print("5. Exit")
        choice = input("Choose an option: ").strip()

        if choice == '1':
            add_expense(expenses)
            save_expenses(expenses)
        elif choice == '2':
            view_expenses(expenses)
        elif choice == '3':
            total_expenses(expenses)
        elif choice == '4':
            monthly_summary(expenses)
        elif choice == '5':
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Loc
a/AI_Coding/expense_tracker.py

Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Monthly Summary
5. Exit
Choose an option: 1
Enter amount: 2000
Enter category (e.g., food, travel): food
Enter description: junk food
Enter date (YYYY-MM-DD) or press Enter for today:
Expense added successfully!

Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Monthly Summary
5. Exit
Choose an option:
```

Explanation:

An expense tracker teaches you how to build real-world software. A basic version loses data and accepts wrong input. By adding validation, saving data, organizing code, and creating summaries, you learn skills used in professional apps: preventing erro This task demonstrates measurable output through score calculation, ensuring that results can be clearly evaluated and tracked. It also incorporates validation to improve user experience by handling inputs effectively and reducing errors. Additionally, it reflects real-world collaboration by using Git workflows such as branching, committing, and merging, which are essential practices in team-based software development.rs, keeping data safe, writing clear code, and analyzing information to find patterns.

Question 2: Version Control Integration

Prompt:

Generate a code for python based file upload script. Identify vulnerabilities such as, No file type validation, Unrestricted file upload.

```
PS D:\veekshitha\AI_Coding> cd D:\veekshitha\AI_Coding\student_grade_calculator; git init
Initialized empty Git repository in D:/veekshitha/AI_Coding/student_grade_calculator/.git/
PS D:\veekshitha\AI_Coding\student_grade_calculator> git add .
PS D:\veekshitha\AI_Coding\student_grade_calculator> git commit -m "Initial commit: Add basic student grade calculator with average calculation"
[master (root-commit) aa4f0a0] Initial commit: Add basic student grade calculator with average calculation
 2 files changed, 44 insertions(+)
 create mode 100644 README.md
 create mode 100644 student_grade_calculator.py
PS D:\veekshitha\AI_Coding\student_grade_calculator> git checkout -b add-grade-calculations
Switched to a new branch 'add-grade-calculations'
PS D:\veekshitha\AI_Coding\student_grade_calculator> git add .
PS D:\veekshitha\AI_Coding\student_grade_calculator> cd D:\veekshitha\AI_Coding\student_grade_calculator; git commit -m "Add grade classification and pe
ulations
>>
>> - Added calculate_percentage function
>> - Added classify_grade function with A-F grading
>> - Updated main function to include percentage and grade class
>> - Updated README with new features"
[add-grade-calculations c7f07aa] Add grade classification and percentage calculations
 2 files changed, 36 insertions(+), 3 deletions(-)
PS D:\veekshitha\AI_Coding\student_grade_calculator> git checkout master
Switched to branch 'master'
PS D:\veekshitha\AI_Coding\student_grade_calculator> git add .; git commit -m "Add license section to README"
[master a5378ae] Add license section to README
 1 file changed, 5 insertions(+), 1 deletion(-)
PS D:\veekshitha\AI_Coding\student_grade_calculator> git merge add-grade-calculations
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS D:\veekshitha\AI_Coding\student_grade_calculator> git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
entage calculations
aa4f0a0 Initial commit: Add basic student grade calculator with aentage calculations
aa4f0a0 Initial commit: Add basic student grade calculator with aentage calculations
entage calculations
aa4f0a0 Initial commit: Add basic student grade calculator with average calculation
PS D:\veekshitha\AI_Coding\student_grade_calculator> python student_grade_calculator.py
Welcome to Student Grade Calculator
Enter the number of subjects:
```

Output:

```
PS D:\veekshitha\AI_Coding\student_grade_calculator> python student_grade_calculator.py
Welcome to Student Grade Calculator
Enter the number of subjects: 5
Enter total marks per subject (default 100): 100
Enter marks obtained for subject 1: 90
Enter marks obtained for subject 2: 80
Enter marks obtained for subject 3: 70
Enter marks obtained for subject 4: 60
Enter marks obtained for subject 5: 50
Average Grade: 70.00
Percentage: 70.00%
Grade Classification: C
PS D:\veekshitha\AI_Coding\student_grade_calculator>
```

Explanation:

The student grade calculator project was created to demonstrate practical Python programming skills, including input handling, calculations, and modular functions. It incorporates version control with Git to showcase professional development practices like branching, committing, and conflict resolution. This approach ensures code maintainability, collaboration readiness, and error-free merges in real-world scenarios. The simple design focuses on core functionality while allowing easy extensions for future enhancements.

Question 3: AI for Commit Message Generation

Prompt:

Generate a Python password validation module and then modify it by adding rules like minimum length and special character requirements, and use AI to generate clear and meaningful Git commit messages describing these changes

Code:

```
# Password validation module import re
 def validate_password(password):    #
Check minimum length    if len(password)
< 8:
        return False, "Password must be at least 8 characters long."
    # Check for special characters    if not
re.search(r'[!@#$%^&*()-+]', password):
        return False, "Password must contain at least one special character."
    # Check for uppercase letters    if not re.search(r'[A-
Z]', password):
        return False, "Password must contain at least one uppercase letter."
    # Check for lowercase letters    if not re.search(r'[a-
z]', password):
        return False, "Password must contain at least one lowercase letter."

    # Check for digits    if not re.search(r'\d',
password):
        return False, "Password must contain at least one digit."
         return True, "Password is valid."
# Example usage password = input("Enter a password: ")
is_valid, message = validate_password(password)
print(message)
# Git commit message: "Added password validation rules for minimum length, special characters, uppercase
letters, lowercase letters, and digits."
```

AI Committed Message:

```
feat: Enhance password validation with length and special character rules

- Add minimum length requirement of 8 characters
- Require at least one special character using regex
- Import re module for pattern matching
- Improve security against weak passwords
```

Human Version:

Added strong password validation rules

Explanation:

The password validator module was developed to enhance application security by enforcing strong password policies. It starts with basic validation to ensure passwords are not empty, then adds rules for minimum length and special characters to prevent weak passwords. This approach protects user accounts from common attacks like brute force or dictionary-based hacks. The modular design allows easy integration into login or registration systems for better overall security.

Question 4: Pair Programming Simulation

Prompt:

Generate an HTML feedback form with JavaScript output.

Code:

```python
""" Simple Quiz Program: A quiz with score calculation and input validation
questions = [
    {
        "question": "What is the capital of France?",
        "options": ["A) Paris", "B) London", "C) Berlin", "D) Madrid"],
        "answer": "A"
    },
    {
        "question": "What is 2 + 2?",
        "options": ["A) 3", "B) 4", "C) 5", "D) 6"],
        "answer": "B"
    }
]
def get_valid_answer():
    """ Get a valid answer from user (A, B, C, or D). """
    while True:
        answer = input("Your answer (A/B/C/D): ").strip().upper()
        if answer in ['A', 'B', 'C', 'D']:
            return answer
        else:
            print("Invalid input. Please enter A, B, C, or D.")
def run_quiz():
    score = 0
    total_questions = len(questions)
    for q in questions:
        print(q["question"])
        for option in q["options"]:
            print(option)
        answer = get_valid_answer()
        if answer == q["answer"]:
            print("Correct!")
            score += 1
        else:
            print("Incorrect.")
    percentage = (score / total_questions) * 100
    print(f"Your score: {score}/{total_questions} ({percentage:.1f}%)")
if __name__ == "__main__":
```

Bash:

```bash
PS D:\veekshitha\AI_Coding> cd D:\veekshitha\AI_Coding\quiz_management_system; git init
Initialized empty Git repository in D:/veekshitha/AI_Coding/quiz_management_system/.git/
PS D:\veekshitha\AI_Coding\quiz_management_system> git add .; git commit -m "Initial commit: Add basic quiz management system with questions and answers"
[master (root-commit) c442164] Initial commit: Add basic quiz management system with questions and answers
 1 file changed, 25 insertions(+)
 create mode 100644 quiz_system.py
PS D:\veekshitha\AI_Coding\quiz_management_system> git checkout -b enhance-quiz
Switched to a new branch 'enhance-quiz'
PS D:\veekshitha\AI_Coding\quiz_management_system> cd D:\veekshitha\AI_Coding\quiz_management_system; git add .; git commit -m "Enhance quiz system with score calculation and input valida
on
>>
>> - Add interactive quiz mode with multiple-choice questions
>> - Implement score tracking and percentage calculation
>> - Add input validation to ensure valid answers (A/B/C/D)
>> - Update main function to run quiz instead of just displaying questions"
[enhance-quiz 7acfc2d] Enhance quiz system with score calculation and input validation
 1 file changed, 35 insertions(+), 5 deletions(-)
PS D:\veekshitha\AI_Coding\quiz_management_system> git checkout master; git merge enhance-quiz
 quiz management system with questions and answers
PS D:\veekshitha\AI_Coding\quiz_management_system>
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python
313/python.exe d:/veekshitha/AI_Coding/quiz_program.py
What is the capital of France?
A) Paris
B) London
C) Berlin
D) Madrid
Your answer (A/B/C/D): A
Correct!
What is 2 + 2?
A) 3
B) 4
C) 5
D) 6
Your answer (A/B/C/D): B
Correct!
Your score: 2/2 (100.0%)
PS D:\veekshitha\AI_Coding>
master  79  50
```

Explanation:

This task demonstrates measurable output through score calculation, ensuring that results can be clearly evaluated and tracked. It also incorporates validation to improve user experience by handling inputs effectively and reducing errors. Additionally, it reflects real-world collaboration by using Git workflows such as branching, committing, and merging, which are essential practices in team-based software development.